

DEPLOYING THE K8S AGENT

Supported from Agent version: K8s 22.3+

EKS on AWS Fargate supported from: K8s Agent 23.4.2+ and Management version X SP2

This feature requires an Add-on License

Before you deploy the K8s Agent, verify that you meet the Agent Requirements on [K8s\[agent-requirements-for-containers.html\]](#) .

In this section:

- Deployment Overview[[deploying-the-k8s-agent.html#UUID-15a5363f-d609-9316-f345-bfa11349deae_N1710917055050](#)]
- Add the K8s Agent to your Registry[[deploying-the-k8s-agent.html#UUID-15a5363f-d609-9316-f345-bfa11349deae_procedure-idm4528164119582433400347170008](#)]
- Deploy with Helm[[deploying-the-k8s-agent.html#UUID-15a5363f-d609-9316-f345-bfa11349deae_N1710916640952](#)]
- Deploy on a Hybrid Fargate Cluster[[deploying-the-k8s-agent.html#UUID-15a5363f-d609-9316-f345-bfa11349deae_procedure-idm4654623471945634110326693656](#)]
- Deploy on a Pure Fargate Cluster[[deploying-the-k8s-agent.html#UUID-15a5363f-d609-9316-f345-bfa11349deae_procedure-idm4600296600139234110349704414](#)]
- Deploy to GKE Autopilot[[deploying-the-k8s-agent.html#UUID-15a5363f-d609-9316-f345-bfa11349deae_procedure-idm4590288657574434281058740357](#)]

Deployment Overview

To protect your containers, you first add the K8s Agent images to your container registry. You then use the referenced Helm chart to deploy these images to your K8s worker nodes, or to AWS Fargate or GKE Autopilot:

- s1-agent

Agent containers are deployed as DaemonSet resources to the target cluster. This causes an Agent Pod to be deployed to each Linux-based node in the cluster. It also ensures that, in the event of a cluster scale-out event, an Agent Pod is automatically scheduled to new cluster nodes. The Agent container monitors events, enforces policies, and communicates with the Singularity Operations Center.

- s1-helper

Helper containers are deployed as a resource with a ReplicaSet of 1. This causes a single Pod to be deployed to a single Linux-based node of the cluster. The helper gathers Kubernetes context from the K8s API on behalf of the Agent. This contextual information is added to all alerts and telemetry data from K8s Agents.

To add the K8s Agent images to your registry:



Note

An alternative method is to use the SentinelOne public repository. For more information, see [How to Install the Agent from a Public Repository\[how-to-install-the-agent-from-a-public-repository.html\]](#).

- 1 Download the Agent package from the Singularity Operations Center.
- 2 Extract the contents of the package. Run

```
docker load -i SentinelAgent_k8s_<AGENT_VERSION>.gz
cwpp_agent/s1helper:<AGENT_VERSION>-x86_64
cwpp_agent/s1agent:<AGENT_VERSION>-x86_64
cwpp_agent/s1helper:<AGENT_VERSION>-aarch64
cwpp_agent/s1agent:<AGENT_VERSION>-aarch64
```



3 Tag the images. Example:

```
docker tag cwpp_agent/s1agent:ga-22.3.3-x86_64 <repository>/s1agent:ga-22.3.3-x86_64
docker tag cwpp_agent/s1helper:ga-22.3.3-x86_64 <repository>/s1helper:ga-22.3.3-x86_64
docker tag cwpp_agent/s1agent:ga-22.3.3-aarch64 <repository>/s1agent:ga-22.3.3-aarch64
docker tag cwpp_agent/s1helper:ga-22.3.3-aarch64 <repository>/s1helper:ga-22.3.3-aarch64
```



4 Push the images to the registry/repository.

```
docker push <repository>/s1agent:ga-22.3.3-x86_64
docker push <repository>/s1helper:ga-22.3.3-x86_64
docker push <repository>/s1agent:ga-22.3.3-aarch64
docker push <repository>/s1helper:ga-22.3.3-aarch64
```



5 (Optional) Create and push image indexes.

```
docker manifest create <repository>/s1agent:ga-22.3.3 \
  --amend <repository>/s1agent:ga-22.3.3-x86_64 \
  --amend <repository>/s1agent:ga-22.3.3-aarch64
docker manifest create <repository>/s1helper:ga-22.3.3 \
  --amend <repository>/s1helper:ga-22.3.3-x86_64 \
  --amend <repository>/s1helper:ga-22.3.3-aarch64
```



```
docker manifest push <repository>/s1agent:ga-22.3.3
docker manifest push <repository>/s1helper:ga-22.3.3
```



The s1-agent and s1-helper images (and indexes) are in an accessible registry. Next, deploy the images with Helm.

To deploy the Agent with Helm:

1 Create a dedicated namespace for the resources that Helm deploys.

```
kubectl create namespace sentinelone
```



2 Configure a pull secret for the namespace to enable the Agent to pull the images. For more information, see [How to create a site key secret\[how-to-create-a-site-key-secret.html\]](#).

3 Add the SentinelOne Helm chart repository:

```
helm repo add sentinelone https://charts.sentinelone.com
```



4 Deploy the Helm chart and reference the prepared container images.

```
helm upgrade --install ${HELM_RELEASE_NAME} \
--namespace=${S1_NAMESPACE} \
--version ${HELM_RELEASE_VERSION} \
--set secrets.site_key.value=${S1_SITE_TOKEN} \
--set configuration.repositories.agent=${REPO_S1AGENT} \
--set configuration.tag.agent=${S1_AGENT_TAG} \
--set configuration.repositories.helper=${REPO_S1HELPER} \
--set configuration.tag.helper=${S1_HELPER_TAG} \
--set secrets.imagePullSecret=${S1_PULL_SECRET_NAME} \
--set configuration.cluster.name=${CLUSTER_NAME} \
sentinelone/s1-agent
```



Note

To set the platform type for Openshift, use the `configuration.platform.type=openshift` parameter in the chart.

To deploy the Agent on a hybrid Fargate cluster:

A hybrid Fargate cluster contains both self-managed K8s worker nodes and Fargate profiles. You deploy the K8s Agent as a DaemonSet on each K8s node. On EKS Fargate, the Agent is injected into each Fargate pod running in a namespace. The helper pod is deployed on one of the nodes.

1 Make sure you meet these prerequisites for deployment to a hybrid Fargate cluster:

- use Agent 23.4.2 GA, or later
- a dedicated non-Fargate namespace for SentinelOne deployment.
- a label added to each Fargate namespace that is monitored by the SentinelOne Agent, except for the SentinelOne deployment namespace.

```
kubectl label namespace {namespace_name} agent-injection-enabled=true
```



- **Note**

If there are workloads in the namespace that you want to exclude from agent injection, add the label `agent-injection-enabled=false` to the pod. Run:

```
kubectl label pods {podname} agent-injection-enabled=false
```



- create a pull secret in each of the namespaces that you want to be monitored by a SentinelOne Agent.

- **Note**

If you are using integrated authentication for the private repo, you do not need to create a pull secret in each namespace.

2 Run the modified helm command with these options:

The helm command in the K8s Agent deployment applies for hybrid clusters as well with the additional configuration setting to allow injection.

```
helm upgrade --install ${HELM_RELEASE_NAME} \
--namespace=${S1_NAMESPACE} \
--version ${HELM_RELEASE_VERSION} \
--set secrets.site_key.value=${S1_SITE_TOKEN} \
--set configuration.repositories.agent=${REPO_S1AGENT} \
--set configuration.tag.agent=${S1_AGENT_TAG} \
--set configuration.repositories.helper=${REPO_S1HELPER} \
--set configuration.tag.helper=${S1_HELPER_TAG} \
--set secrets.imagePullSecret=${S1_PULL_SECRET_NAME} \
--set configuration.env.injection.enabled=true \
--set configuration.cluster.name=${CLUSTER_NAME} \
sentinelone/s1-agent
```



To deploy the Agent on a pure Fargate cluster:

A pure Fargate cluster does not have any self-managed K8s worker nodes, but only Fargate profiles. The EKS Fargate Agent is deployed to every Fargate pod in a specific namespace labeled with **agent-injection-enabled=true**. A Helper pod is also deployed in a separate namespace without this label to prevent self-injection.

1 Make sure you meet these prerequisites for deployment to a pure Fargate cluster:

- use Agent 23.4.2 GA, or later
- a dedicated Fargate profile and namespace for SentinelOne deployment.
- a label added to each Fargate namespace that is monitored by the SentinelOne Agent, except for the SentinelOne deployment namespace.

```
kubectl label namespace {namespace_name} agent-injection-enabled=true
```



Note

If there are workloads in the namespace that you want to exclude from agent injection, add the label `agent-injection-enabled=false` to the pod. Run:

```
kubectl label pods {podname} agent-injection-enabled=false
```



- create a pull secret in each of the namespaces that you want to be monitored by a SentinelOne Agent.



Note

If you are using integrated authentication for the private repo, you do not need to create a pull secret in each

namespace.

2 Run the modified helm command with these options:

The helm command in K8s Agent deployment applies for a pure cluster with additional configuration settings to deploy the `s1-helper` as a Fargate pod and allow injection.



Note

The `helper.labels` values must match the pod selector label set for the SentinelOne namespace in a dedicated Fargate profile.

```
helm upgrade --install ${HELM_RELEASE_NAME} \
--namespace=${S1_NAMESPACE} \
--version ${HELM_RELEASE_VERSION} \
--set secrets.site_key.value=${S1_SITE_TOKEN} \
--set configuration.repositories.agent=${REPO_S1AGENT} \
--set configuration.tag.agent=${S1_AGENT_TAG} \
--set configuration.repositories.helper=${REPO_S1HELPER} \
--set configuration.tag.helper=${S1_HELPER_TAG} \
--set secrets.imagePullSecret=${S1_PULL_SECRET_NAME} \
--set configuration.env.injection.enabled=true \
--set helper.labels.${S1_NAMESPACE_SELECTOR_KEY}=${S1_NAMESPACE_SELECTOR_VALUE} \
--set configuration.cluster.name=${CLUSTER_NAME} \
sentinelone/s1-agent
```



To deploy the Agent to GKE Autopilot

Supported from Agent version K8s 24.1+

GKE Autopilot automatically manages your cluster configuration, including nodes, scaling, security, and other preconfigured settings. To monitor and detect security events in your GKE clusters, deploy the K8s Agent to GKE Autopilot.

1 Follow the steps in [How to Install the Agent from a Public Repository](#)[[how-to-install-the-agent-from-a-public-repository.html](#)] to deploy the Agent using a Helm chart.



Note

For GKE Autopilot, only pull images from the public repo.

- 2 When you run the helm deployment command, include this flag to enable GKE Autopilot:

```
--set configuration.platform.gke.autopilot=true
```



Note

When GKE Autopilot is enabled, the Agent runs in read-only mode. Threat detection is enabled, and alerts flow to the Singularity Operations Center, but mitigation is not supported.

During the helm deployment, the following values are applied:

- The value of `requests resources` is set to equal the value of `limits resources` . This meets a requirement of GKE Autopilot.
- The environment variable `S1_GKE_AUTOPILOT` is set to `true` .
- The value of `ephemeral-storage` is set to 1Gi to meet the recommended storage for the Agent. You can change this value if necessary.

For an example configuration, see [Helm Chart Options\[k8s-agent-helm-chart-options.html\]](#).

Was this helpful?

Yes

No

© 2025 SentinelOne